

FabricaLog

Documentação Técnica Completa

Controle de produção e ponto de funcionários
Forno CEDAN · Bela Cruz – CE

Versão 1.2 · Maio de 2026

Stack: React 19 · Vite 8 · PWA · Capacitor · localStorage · Supabase

Índice

1.	Contexto e Objetivo
2.	Decisões de Stack
3.	Estrutura de Arquivos
4.	Design System
5.	Gerenciamento de Estado
6.	Persistência — storage.js
7.	Módulo Dashboard
8.	Módulo Semana
9.	Módulo Ponto
10.	Módulo Equipe
11.	Configurações
12.	Utilitários
13.	Exportação XLSX
14.	Geração de Recibos PDF
15.	Backup e Restauração
16.	PWA e Modo Offline
17.	Distribuição Android (Capacitor)
18.	Módulo Cargas
19.	Banco de Dados — Supabase

1. Contexto e Objetivo

O **Forno CEDAN** é uma olaria localizada em Bela Cruz – CE que produz tijolos cerâmicos. Antes do FabricaLog, a operação era controlada por **duas planilhas Excel**:

- Controle Semanal de Produção: registra queima, enforma, qualidade do tijolo, estoque, vendas, funcionários e ocorrências por dia da semana.
- Controle de Ponto: registra o valor diário de cada um dos ~22 funcionários, falta (F), bônus de assiduidade e total a pagar por semana.

O FabricaLog substitui essas planilhas por um **aplicativo web offline** que roda no celular do gerente de produção, sem depender de internet, servidor ou nuvem. Todos os dados ficam no próprio dispositivo. A exportação para Excel preserva a compatibilidade com o fluxo atual de envio ao contador.

DECISÃO: Aplicativo local, não SaaS

Por quê: A fábrica não tem internet estável. O gerente precisa registrar dados em tempo real durante o turno. Dependendo de uma API externa criaria indisponibilidade crítica.

Impacto: Todos os dados vivem em localStorage no dispositivo. Não há servidor, não há autenticação, não há custo de infraestrutura mensal.

2. Decisões de Stack

2.1 React 19 + Vite 8

React foi escolhido por ter o maior ecossistema de componentes UI, suporte de primeira classe ao CSS Modules e familiaridade ampla. O Vite 8 oferece build com HMR instantâneo e o plugin oficial **vite-plugin-pwa** para geração automática do service worker Workbox.

DECISÃO: React em vez de Vue ou Svelte

Por quê: O projeto pode crescer (relatórios, gráficos, múltiplas unidades). React tem cobertura de biblioteca mais ampla e o time de manutenção já conhece a API.

Impacto: Nenhum compilador extra, JSX é direto e CSS Modules funciona nativamente com Vite.

2.2 CSS Modules

Cada componente/módulo tem seu próprio arquivo **.module.css**. Os seletores são transformados em hashes únicos pelo Vite em tempo de build, eliminando colisão de nomes globais sem nenhuma biblioteca de CSS-in-JS.

DECISÃO: CSS Modules em vez de Tailwind ou styled-components

Por quê: Tailwind v4 + Vite 8 apresentou incompatibilidades com classes arbitrárias durante o desenvolvimento. CSS Modules é zero-runtime, sem purge e sem configuração extra. Cada decisão de estilo fica no próprio arquivo do componente, co-localizada.

Impacto: Build 100% estático, sem runtime de CSS, com escopo automático por arquivo.

2.3 localStorage como banco de dados

Toda a persistência usa **localStorage** via um adapter centralizado (**storage.js**). A chave de prefixo **fabricalog_v1** isola os dados do app de qualquer outro dado salvo no navegador e abre espaço para migração de schema futura (v2, v3...).

DECISÃO: localStorage em vez de IndexedDB ou SQLite

Por quê: O volume de dados é pequeno (semanas de produção, ~22 funcionários). localStorage suporta até 5–10 MB por origem. A API é síncrona e simples. IndexedDB seria overengineering para o problema. SQLite via WASM adicionaria ~500 KB ao bundle.

Impacto: Quando a escala exigir SQLite (múltiplas unidades, histórico de anos), apenas o arquivo storage.js precisa mudar — todos os stores continuam iguais.

2.4 jsPDF para geração de recibos

Os recibos de pagamento são gerados **no browser**, sem servidor, usando **jsPDF 4.x**. O import é lazy (await import('jspdf')) para não afetar o tempo de carregamento inicial do app.

DECISÃO: jsPDF em vez de html2canvas ou pdfmake

Por quê: html2canvas renderiza um screenshot HTML para canvas e depois para PDF, produzindo imagens de baixa resolução e arquivos grandes. pdfmake usa um DSL de layout próprio mas tem bundle maior (~800 KB). jsPDF com layout por coordenadas dá controle total sobre cada milímetro do documento, como o ReportLab no V1 Python.

Impacto: Recibos vetoriais, pequenos, compatíveis com qualquer leitor de PDF.

2.5 SheetJS (xlsx) para exportação Excel

A exportação para planilha usa **SheetJS (xlsx 0.18.x)**, também com import lazy. A função **XLSX.utils.aoa_to_sheet()** constrói as planilhas a partir de arrays de arrays, exatamente como o modelo original das planilhas do gerente.

DECISÃO: SheetJS em vez de ExcelJS

Por quê: SheetJS tem melhor suporte a .xlsx no browser sem dependências nativas. ExcelJS funciona bem em Node.js mas seu bundle para browser é maior e menos estável. SheetJS é o padrão de fato para geração de Excel client-side.

Impacto: Output compatível com Excel, LibreOffice e Google Sheets sem conversão.

2.6 PWA com Workbox

O **vite-plugin-pwa** injeta um service worker Workbox que pré-caches todos os assets estáticos (JS, CSS, HTML, fontes, ícones) no momento da instalação. Após a primeira carga, o app roda 100% offline.

DECISÃO: PWA em vez de Tauri ou Electron

Por quê: Tauri e Electron empacotam um browser inteiro junto com o app. Para o caso de uso (celular Android do gerente), um APK via Capacitor com os arquivos da PWA embutidos é muito mais leve (~5 MB vs ~80 MB para Electron). Tauri no Android ainda era experimental no período de desenvolvimento.

Impacto: PWA + Capacitor: build web único que funciona no browser e como APK nativo.

3. Estrutura de Arquivos

A estrutura segue separação por responsabilidade — componentes genéricos em **components/**, lógica de negócio agrupada por módulo em **modules/**, estado em **store/**, e funções puras em **utils/**.

src/	Raiz do código-fonte
■■■■ main.jsx	Entry point — monta React, registra SW
■■■■ App.jsx	Shell de layout, roteamento por módulo, integração dos stores
■■■■ index.css	Design tokens (CSS custom properties) + reset + font Syne
■■■■ components/	Componentes genéricos reutilizáveis
■ ■■■■ Ic.jsx	Dispatcher de ícones SVG inline
■ ■■■■ Btn.jsx	Botão: primary ghost danger
■ ■■■■ Modal.jsx	Overlay via createPortal + visual viewport tracking
■ ■■■■ WeekNavigator.jsx	Navegação ← Semana N →
■ ■■■■ BottomNav.jsx	Barra de nav inferior (mobile)
■ ■■■■ Sidebar.jsx	Barra lateral (desktop ≥ 768 px)
■■■■ modules/	Módulos de negócio, cada um auto-contido
■ ■■■■ dashboard/	Dashboard: resumo da semana selecionada
■ ■■■■ semana/	Controle semanal de produção
■ ■■■■ ponto/	Controle de ponto e cálculo de pagamentos
■ ■■■■ equipe/	Gestão de funcionários
■ ■■■■ configuracoes/	Dados da empresa para os recibos
■ ■■■■ carga/	Registro de carregamentos — caminhão empresa/externo
■ ■■■■ dashboard/	já existe — atualizado com DashboardPlano e FornosChart
■■■■ store/	Custom hooks de estado com persistência
■ ■■■■ storage.js	Adapter localStorage com debounce
■ ■■■■ useSemanaStore.js	CRUD das semanas de produção
■ ■■■■ usePontoStore.js	CRUD dos pontos semanais
■ ■■■■ useEmployeeStore.js	Gestão do roster de funcionários
■ ■■■■ useSettingsStore.js	Configurações da empresa
■■■■ data/	Dados estáticos e seed de desenvolvimento

■ ■■■ employees.js	Array fixo com os 22 funcionários e IDs estáveis
■ ■■■ seed.js	Dados reais de 4 semanas (só disponível em DEV)
■■■ utils/	Funções puras sem efeitos colaterais
■■■ calcPonto.js	Calcula dias trabalhados, bônus e total por funcionário
■■■ calcSemana.js	Conta total de fornos enfiados na semana
■■■ weekLabel.js	Formata label da semana e detecta semana passada
■■■ formatBRL.js	Formata valores como R\$ 1.250,00
■■■ valorPorExtenso.js	Converte R\$ para "mil duzentos e cinquenta reais"
■■■ gerarRecibos.js	Gera PDF de recibos com jsPDF
■■■ backup.js	Exporta e importa JSON de backup
■■■ calcVariavel.js	Calcula variável do gerente: milheiros + bônus de fornos

4. Design System

Todo o design system é declarado como **CSS custom properties** em **index.css**. Nenhum componente usa valores de cor ou espaçamento hardcoded — tudo referencia variáveis, o que permite futura troca de tema sem tocar em nenhum módulo.

4.1 Paleta de Cores — espaço oklch

A paleta usa o espaço de cor **oklch()**, introduzido no CSS Color Level 4. oklch é perceptualmente uniforme: mudar o parâmetro L (luminosidade) em um incremento fixo produz o mesmo salto visual em qualquer matiz, o que torna as variações de estado (hover, disabled, active) previsíveis e consistentes.

--bg	oklch(12% 0.016 38)	Fundo da página — marrom escuro quase preto
--bg-card	oklch(17% 0.018 38)	Fundo de cards — um degrau acima do bg
--bg-nav	oklch(10% 0.014 38)	Fundo da sidebar/bottomnav — mais escuro
--accent	oklch(65% 0.19 38)	Laranja queimado — cor de identidade
--accent-dim	oklch(65% 0.19 38 / 0.15)	Versão translúcida do accent (hover, active)
--text	oklch(93% 0.008 38)	Texto primário — quase branco com tom quente
--text-dim	oklch(65% 0.012 38)	Texto secundário — labels, captions
--success	oklch(68% 0.16 148)	Verde — meta atingida, vendas
--warning	oklch(76% 0.17 68)	Âmbar — alerta, estoque
--danger	oklch(63% 0.21 22)	Vermelho — exclusão, falta (F no ponto)
--border	oklch(22% 0.018 38)	Bordas sutis entre cards
--border-mid	oklch(30% 0.02 38)	Bordas de maior destaque
--radius-sm	6px	Arredondamento pequeno — botões, inputs
--radius-md	12px	Cards médios, pills
--radius-lg	20px	Cards principais, modals

4.2 Tipografia — Syne

A fonte **Syne** (Google Fonts) foi escolhida por ter variação de peso de 400 a 800, o que permite hierarquia tipográfica clara sem misturar famílias. O peso 800 é usado em valores principais (totais, números de destaque). Letter-spacing negativo (**-0.03em** a **-0.04em**) em números grandes cria a densidade visual característica de interfaces de dados premium (Linear, Vercel).

4.3 Layout Responsivo

Breakpoint único em **768 px**, declarado em **index.css** com media query **@media (min-width: 768px)**. Abaixo do breakpoint, o **BottomNav** fica fixo no rodapé e o conteúdo tem **padding-bottom: 80px** para não ser ocultado. Acima do breakpoint, a **Sidebar** aparece com **220 px** de largura fixa e o conteúdo tem **padding-left: 260 px**.

DECISÃO: Breakpoint único em vez de múltiplos (sm, md, lg)

Por quê: O app tem um único usuário primário em um único dispositivo. Breakpoints adicionais adicionam complexidade sem benefício real.

Impacto: CSS mais simples, mais fácil de manter.

5. Gerenciamento de Estado

O estado é gerenciado por **custom hooks React**, sem biblioteca externa (Redux, Zustand, Jotai). Cada store é um hook que expõe o array de itens e as operações permitidas. Os stores são instanciados em **App.jsx** e passados como props até três níveis de profundidade — prop drilling aceitável para o tamanho atual do app.

DECISÃO: Custom hooks em vez de Zustand ou Context API

Por quê: Zustand seria a escolha certa se o app crescesse para 10+ componentes compartilhando o mesmo estado. No estado atual, a Context API gera re-renders desnecessários sem memoização cuidadosa, e Zustand é uma dependência extra. Custom hooks são transparentes, testáveis e têm zero overhead.

Impacto: Se o app crescer, a migração para Zustand é trivial: o hook mantém a mesma assinatura, apenas a implementação interna muda.

5.1 Padrão comum dos stores

Todos os quatro stores seguem o mesmo padrão:

- **useState** inicializado com leitura do localStorage (lazy init — executado uma vez)
- **useRef(false)** como flag "mounted" para ignorar o primeiro efeito de persistência e evitar gravar no storage com dados recém-lidos do storage
- **useEffect** que persiste quando items muda, pulando o primeiro render
- Operações: **create**, **update**, **remove**, **replaceAll**, **getById**

5.2 useSemanaStore

Gerencia o array de semanas de produção. Cada semana tem:

id	UUID gerado com <code>crypto.randomUUID()</code> — estável, sem colisão
numero	Número sequencial da semana (1, 2, 3...)
dataInicio	Data ISO da segunda-feira da semana (ex: "2026-04-06")
meta	Meta de fornos enformados por semana (padrão: 10)
dias	Array de 6 objetos <code>EMPTY_DIA</code> (seg → sáb)

A operação **updateDia(id, diaIndex, patch)** atualiza um único dia imutavelmente usando dois levels de map, evitando mutação do array original.

5.3 usePontoStore

Gerencia os pontos semanais. O campo **dias** é um **objeto** (não array), indexado pelo ID do funcionário. Isso permite que funcionários sejam adicionados ou removidos sem quebrar índices numéricos.

Cada entrada **dias[empId]** tem as chaves **seg, ter, qua, qui, sex, sab** (valor numérico ou "F" para falta) e opcionalmente **seg_nota, ter_nota...** para notas de dia com valor atípico.

5.4 useEmployeeStore

Gerencia o roster de funcionários. Inicia com os 22 funcionários do arquivo **employees.js** quando não há dados no localStorage. Permite adicionar, renomear e remover. IDs são UUIDs — os funcionários originais têm IDs fixos (e1, e2... e22) para compatibilidade com backups antigos.

5.5 useSettingsStore

Gerencia os dados da empresa usados nos recibos. Tem valores padrão hardcoded (nome da empresa, CNPJ, endereço, cidade) que são sobrescritos pelo que estiver no localStorage. Isso permite que o app funcione sem nenhuma configuração inicial.

6. Persistência — storage.js

O arquivo **storage.js** é o único ponto de contato com o `localStorage`. Ele implementa três operações: **load**, **save** e **clear**.

6.1 Prefixo de namespace

Todas as chaves são prefixadas com **fabricalog_v1_**. O prefixo tem dois objetivos:

- Isolar os dados do app de outros dados que possam estar no `localStorage` do mesmo domínio (por exemplo, Vite salva configurações de HMR).
- Versionar o schema: quando a estrutura dos dados mudar de forma incompatível, o prefixo muda para **fabricalog_v2_** e o código de migração pode ler a versão antiga e escrever na nova.

6.2 Debounce no save

O **save** usa **setTimeout** de 400ms com **clearTimeout** no início. O efeito prático é: se o usuário digitar em um campo de formulário, cada keystroke agenda um novo save e cancela o anterior. O `localStorage` só é escrito **400ms após o último keystroke**.

DECISÃO: Debounce de 400ms no save

Por quê: Sem debounce, cada digitação em `DiaForm` acionaria uma serialização JSON e uma gravação em disco (via `localStorage`). Em dispositivos Android lentos, isso pode causar janks visíveis na UI.

Impacto: O save real acontece poucos instantes após o usuário parar de digitar, sendo imperceptível e seguro em caso de fechamento abrupto do app.

6.3 Tratamento de erros

O **load** envolve o parse JSON em `try/catch` e retorna o fallback em qualquer erro (JSON corrompido, chave inexistente, `localStorage` bloqueado pelo browser). O **save** também tem `try/catch` e loga o erro apenas em desenvolvimento, nunca quebrando a UI em produção.

7. Módulo Dashboard

O Dashboard é a tela inicial do app. Exibe um resumo da semana selecionada com navegação entre semanas passadas, gráfico de produção diária, gráfico de fornos, resumo operacional de picos e uma aba de plano semanal.

7.1 WeekNavigator

O array de semanas é ordenado decrescente por número e o índice de seleção começa em 0 (semana mais recente). O WeekNavigator incrementa/decrementa o índice. Os botões são desabilitados nos extremos do array.

7.2 Hero Card — Total Produzido

A métrica principal é o **Total Produzido** em milheiros (vendas + estoque). A decisão de somar os dois campos responde à realidade operacional: o gerente precisa saber quanto foi *tirado do forno*, independentemente de ter sido vendido ou ficado em estoque. Vendas e estoque aparecem como sub-métricas abaixo do total principal, em cores distintas (verde para vendas, âmbar para estoque).

7.3 Card de Fornos / Meta

O card de meta exibe o contador de fornos enformados na semana vs. a meta definida. Tem quatro elementos visuais:

- **Badge de status:** "✓ meta atingida" (verde) ou "faltam N" / "faltou N" (âmbar)
- **Número grande:** o total de fornos em 52px, peso 800
- **Barra de progresso:** fill verde se meta atingida, accent se não
- **Dots:** um círculo por forno da meta, preenchido verde até o total atingido

A distinção "**faltam**" vs "**faltou**" é determinada pela função **isWeekPast(semana)** — se o sábado da semana já passou, o verbo é pretérito. Isso é importante para o gerente entender se ainda pode recuperar a meta ou se a semana já encerrou abaixo dela.

7.4 BarChart — SVG puro

O gráfico de barras usa **SVG inline** sem nenhuma biblioteca de charting. Cada dia tem uma barra com **3 segmentos empilhados**: estoque (base, verde), galpão (meio, azul) e vendas (topo, laranja). Um total numérico é renderizado acima de cada barra. O SVG usa **viewBox="0 0 600 200"** com **width="100%"** para ser responsivo.

DECISÃO: SVG puro em vez de Recharts ou Chart.js

Por quê: Recharts adiciona ~250 KB ao bundle. Chart.js adiciona ~200 KB. Para um único tipo de gráfico com dados simples, uma implementação SVG de ~80 linhas é mais eficiente, sem dependências externas e totalmente customizável.

Impacto: Bundle menor, sem re-renders por biblioteca, comportamento 100% previsível.

7.5 FornosChart — SVG puro

Gráfico de barras SVG puro exclusivo para fornos enformados por dia. Barras laranjas quando a meta é atingida, amarelas quando abaixo. Uma linha tracejada por barra indica a meta do dia (do plano semanal). Mostra o valor numérico acima de cada barra.

7.6 Resumo Operacional

Seção com 4 estatísticas de pico da semana: **Pico de vendas** (dia + valor em milh.), **Maior estoque** (dia + valor), **Maior enformamento** (dia + nº de fornos), **Equipe máxima** (dia + nº de funcionários). Cada peak é calculado com **Array.reduce** sobre os 6 dias.

7.7 Aba Plano (DashboardPlano)

Aba separada que exibe o plano de fornos por dia vs. realizado. Tem um card de progresso de fornos (total semanal vs meta), um card de progresso de milheiros (se **metaMilheiros > 0**), um callout do dia atual com meta ajustada, e um grid de 6 cards por dia. O campo **metaMilheiros** é a meta diária de milheiros (editável pelo gerente). O campo **plano** é um array de 6 inteiros distribuindo a meta de fornos pelos dias. A função **calcAjustado** redistribui os fornos restantes proporcionalmente quando dias passados ficaram abaixo/acima do plano.

7.8 Exportação do Dashboard como PDF

A função **exportDashboard(semana, prevSemana)** em **dashboardExport.js** gera um PDF de fechamento semanal usando **html2canvas + jsPDF**. O HTML é montado inline com estilos inline (sem `oklch` — somente hex, pois `html2canvas` não renderiza `oklch`). O layout inclui: topbar com logo, KPI cards (total, funcionários, fornos), barra de meta, card Variável (cálculo do variável do gerente: vendas × R\$1,00 + estoque × R\$0,25 + bônus de fornos por faixa), gráficos de barras por dia (vendas, estoque, funcionários, fornos), ocorrências e resumo operacional. A função **exportDashboardMes** gera relatório mensal equivalente.

8. Módulo Semana

O módulo Semana tem duas telas: a lista de semanas (**SemanaList**) e o detalhe de uma semana (**SemanaDetalhe** + **DiaForm**).

8.1 SemanaList

Exibe cards de semanas ordenados do mais recente ao mais antigo. Cada card mostra o label da semana, o total de milheiros, o progresso de fornos e o status da meta. Tem um botão de criação de nova semana com modal simples pedindo número e data de início.

8.2 SemanaDetalhe

Tela de detalhe com:

- **Header:** botão voltar, título com weekLabel, sub-título com meta
- **Barra de progresso:** fill linear da meta
- **Totals row:** Total Milheiros (V: X · E: X) e Fornos enformados
- **Pills de dia:** Seg – Sáb, scroll horizontal com mask-image fade
- **DiaForm:** formulário do dia ativo
- **Modals:** exclusão da semana e edição da meta

8.3 DiaForm — campos e estrutura

O formulário de cada dia tem os seguintes campos, espelhando exatamente a planilha original:

queima	text	Número do forno que está em queima no dia
enfornas[]	text ×3	Até 3 fornos enformados no dia (array de 3 posições)
qualidade	select	Qualidade do tijolo: Pátio Seco / Pátio Molhado / Secador
estoque	number	Milheiros que entraram para o estoque no dia
vendas	number	Milheiros vendidos no dia
fornosDesocupados	text	Fornos disponíveis para enformamento
reforma	text	Reformas realizadas no dia
qtdFunc	number	Quantidade de funcionários trabalhando
ocorrendia	textarea	Ocorrências, incidentes, observações do dia
emCaminhoes	number	Milheiros no galpão — saída do forno não contabilizada em vendas/estoque

8.4 Contagem de fornos — calcSemana.js

A função **countFornos(dias)** conta o total de fornos enformados na semana. Cada dia tem um array **enfornas[]** de até 3 strings. Um forno é contado se a string não estiver vazia após `trim()`. O resultado alimenta a barra de progresso de meta tanto no Dashboard quanto no SemanaDetalhe.

8.5 Exportação para Excel — `semanaExport.js`

A função **exportSemana(semana)** constrói a planilha como array de arrays (AOA) e usa **XLSX.utils.aoa_to_sheet()**. As primeiras 3 linhas são título e período, seguidas pelo header e pelos 10 campos de dados. Há uma linha de totais para campos numéricos e uma linha separada para a meta. As larguras de coluna são definidas com **ws['!cols']**.

9. Módulo Ponto

O módulo Ponto replica o controle de ponto da planilha original. Exibe uma tabela com uma linha por funcionário e uma coluna por dia da semana.

9.1 Estrutura de dados do ponto

O objeto **dias** de um ponto é indexado por ID de funcionário, não por posição numérica. Isso garante que adicionar ou remover funcionários não desloca os dados de outros.

Cada entrada tem as chaves **seg, ter, qua, qui, sex, sab**. O valor pode ser:

- "F" — falta (visualizado com fundo vermelho na célula)
- número como string — valor em R\$ do dia trabalhado (ex: "100", "55")
- "" — célula vazia (dia não preenchido)
- Opcionalmente: **seg_nota, ter_nota...** para notas textuais

9.2 calcPonto.js — cálculo por funcionário

A função **calcPonto(diasObj)** é uma função pura que recebe o objeto de dias de um funcionário e retorna três valores:

dias	Número de dias trabalhados (células que não são "F" e não estão vazias)
bonus	R\$ 25 se dias >= 6, caso contrário 0 — bônus de assiduidade e produção
total	Soma de wages (valores numéricos dos dias) + bonus

DECISÃO: Bônus de R\$ 25 por 6 dias trabalhados

Por quê: Regra operacional da empresa. O funcionário que trabalhar os 6 dias da semana (seg–sáb) recebe R\$ 25 extras. A semana de ponto vai de segunda a sábado.

Impacto: A regra está encapsulada em calcPonto.js. Se mudar, muda em um único lugar.

9.3 PontoCellEditor — edição inline

Ao clicar em qualquer célula da tabela, um popover inline abre sobre a célula. O popover tem um input numérico, um botão "F" para marcar falta e um campo opcional de nota. Fecha ao clicar fora (useEffect com listener no document). A edição é imediata — não há botão de confirmação separado.

DECISÃO: Popover inline em vez de modal global

Por quê: Um modal centralizado forçaria o usuário a perder o contexto da linha que está editando. O popover aparece sobre a célula, mantendo a referência visual de qual funcionário e dia estão sendo editados.

Impacto: UX mais rápida para preenchimento de uma tabela densa.

9.4 Exportação para Excel — `pontoExport.js`

A função `exportPonto(ponto, employees)` gera uma planilha com:

- Header com metadados (período, legenda de F e bônus)
- Uma linha por funcionário com dias, bônus e total
- Seção de notas (apenas para funcionários com notas preenchidas)
- Linha de totais gerais (TOTAL GERAL) no rodapé

10. Módulo Equipe

O módulo Equipe exibe e gerencia o roster de 22 funcionários. Permite adicionar novos funcionários, renomear e remover (com confirmação). Ao criar um novo ponto, o store lê a lista atual de funcionários para gerar o objeto **dias** com entradas para cada ID.

10.1 IDs fixos vs. UUIDs dinâmicos

Os 22 funcionários originais têm IDs curtos fixos (**e1**, **e2**... **e22**). Funcionários adicionados pelo usuário recebem UUIDs gerados com **crypto.randomUUID()**. A razão dos IDs fixos é compatibilidade: backups exportados antes de qualquer alteração de nome podem ser reimportados sem perda de vínculo entre funcionário e dados de ponto.

10.2 Funcionários removidos em pontos antigos

Se um funcionário é removido da equipe mas existem pontos antigos com dados para o ID dele, esses dados permanecem no objeto **dias** do ponto mas não são exibidos (o componente itera sobre **employees** ativos, não sobre as chaves de **dias**). Os dados não são destruídos — ficam no `localStorage` como dados órfãos preservados.

11. Configurações

O módulo de configurações (**ConfigModal.jsx**) permite editar os dados da empresa que aparecem nos recibos de pagamento. É acessado pelo botão de engrenagem na Sidebar (desktop) ou no menu de ações mobile.

11.1 Dados configuráveis

empresa	Razão social da empresa (aparece no texto do recibo)
cnpj	CNPJ formatado (aparece no texto do recibo)
endereco	Endereço completo (aparece no texto do recibo)
cidade	Cidade (aparece na linha de data do recibo)

11.2 Valores padrão

O **useSettingsStore** tem os dados da empresa CEDAN como valores padrão. Isso garante que, mesmo sem nenhuma configuração, os recibos são gerados corretamente na primeira instalação. Ao salvar, o **localStorage** sobrepõe os padrões.

12. Utilitários

12.1 weekLabel.js

Centraliza toda a lógica de formatação de datas de semana. Funções exportadas:

weekLabel(semána)	"Semana 4 — 20/04 a 25/04" — label completo
shortWeekLabel(semána)	"Sem. 4" — label curto para cards
isWeekPast(semána)	True se o sábado da semana já passou (para verbo "faltou")
DAY_NAMES	['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb']
DAY_KEYS	['seg', 'ter', 'qua', 'qui', 'sex', 'sab']
DAY_FULL_NAMES	['Segunda-Feira', 'Terça-Feira'...] — para cabeçalho do xlsx

Detalhe de isWeekPast: a data de início (**dataInicio**) é uma string ISO da segunda-feira. O sábado é calculado somando 5 dias. Comparado com **new Date()** (agora). Se o sábado estiver no passado, retorna true.

12.2 formatBRL.js

Usa **Intl.NumberFormat** com locale **pt-BR** e estilo **currency**. Produz "R\$ 1.250,00". Não reinventa formatação de moeda — delega para a API nativa do browser que já sabe as regras de separadores do PT-BR.

12.3 valorPorExtenso.js

Converte um valor em reais para texto por extenso em Português do Brasil. Implementação própria sem dependência externa, construída sobre três tabelas:

- **UNIDADES:** zero a dezenove (tratamento especial para números ímpares)
- **DEZENAS:** vinte a noventa
- **CENTENAS:** cem a novecentos (com distinção "cem" vs "cento + resto")

A função recursiva **numExtenso(n)** decompõe o número em milhões, milhares, centenas, dezenas e unidades, juntando com " e ". O resultado final inclui o valor formatado em BRL seguido do extenso entre parênteses: "R\$ 125,00 (cento e vinte e cinco reais)".

DECISÃO: Implementação própria em vez de num2words ou similar

Por quê: A dependência **num2words** adiciona >10 KB e requer configuração de locale. A implementação própria tem <50 linhas, cobre todos os casos de uso (reais, centavos, até milhões) e é facilmente testável.

Impacto: Zero dependências, comportamento 100% previsível, fácil de auditar.

12.4 calcVariavel.js

Calcula o variável do gerente a partir dos dados da semana. Recebe **{totalVendas, totalEstoque, totalFornos}**. Retorna **{milheiros, bonusFornos, total}**.

Regras: milheiros = vendas × R\$1,00 + estoque × R\$0,25. Bônus de fornos por faixa: 6–7 = R\$100, 8–9 = R\$150, 10–11 = R\$200, 12+ = R\$300. A regra está encapsulada neste único arquivo.

13. Exportação XLSX

Ambas as exportações (semana e ponto) seguem o mesmo padrão:

- Import lazy: `const XLSX = await import('xlsx')` — SheetJS não é carregado até o primeiro click de exportação
- `XLSX.utils.aoa_to_sheet(rows)` — constrói a planilha a partir de array de arrays
- `ws['!cols'] = [{wch: N}...]` — define largura de colunas
- `XLSX.writeFile(wb, filename)` — dispara download direto no browser

13.1 semanaExport.js — estrutura da planilha

Linhas da planilha de semana:

Linha 1	Título: "CONTROLE SEMANAL DE PRODUÇÃO — FORNO CEDAN"
Linha 2	Período: "Semana: 4 (Semana 4 — 20/04 a 25/04)"
Linha 3	Vazia
Linha 4	Header: CATEGORIA Seg – Sáb TOTAL
Linhas 5–15	Dados: Queima, Enforna 1/2/3, Qualidade, Estoque, Vendas, Fornos Desocp., Reforma, Qtd Func, Ocorrências
Linha 16	Meta: valor da meta na coluna TOTAL

13.2 pontoExport.js — estrutura da planilha

Linhas da planilha de ponto:

Linhas 1–3	Metadados: título, período, legenda (F = Faltas, R\$25 = bônus)
Linha 4	Vazia
Linha 5	Header: #, FUNCIONÁRIO, SEG–SÁB, DIAS, BÔNUS, TOTAL
Linhas 6–N	Uma linha por funcionário
Seção NOTAS	Sublinhas com notas dos dias (aparece apenas se há notas)
Última linha	TOTAL GERAL com soma de todos os campos

14. Geração de Recibos PDF

A geração de recibos é o único componente do app que produz um documento PDF diretamente no browser, usando **jsPDF 4.x**. O objetivo é replicar o formato do recibo físico usado na empresa.

14.1 Layout do recibo

Cada recibo ocupa uma caixa com borda cinza, auto-dimensionada antes do render. A caixa contém:

- **Título** "RECIBO DE PAGAMENTO" (esquerda) + valor formatado em BRL (direita), **negrito**
- **Corpo**: texto corrido com o valor por extenso em **negrito inline**
- **Cidade e data**: ex. "Bela Cruz, 3 de maio de 2026."
- **Linha de assinatura**: 55% da largura interna, centralizada
- **Nome do funcionário**: centralizado abaixo da linha

14.2 O problema do texto com negrito inline — `flowRichText`

jsPDF não tem suporte nativo a parágrafos com mixed style (normal + negrito na mesma linha). A solução implementada usa dois passos:

Passo 1 — `flowRichText(doc, segments, maxWidth)`: recebe um array de segmentos `{text, bold}` e mede cada palavra individualmente com o font correto (`doc.getTextWidth()` após `doc.setFont(bold/normal)`). Divide as palavras em linhas respeitando `maxWidth`. Retorna um array de linhas, cada linha sendo um array de tokens `{text, bold, width}`.

Passo 2 — `drawRichText(doc, lines, x, y, lineHeight)`: itera os tokens de cada linha, alternando `setFont` conforme o flag `bold` do token, e avançando `x` pela largura pré-calculada do token. Retorna o novo `y`.

14.3 Cálculo de altura antes do render — `calcBoxHeight`

jsPDF desenha elementos em ordem, de cima para baixo. Para desenhar a borda da caixa antes do conteúdo, é preciso saber a altura total com antecedência. `calcBoxHeight(func)` executa `flowRichText` sem chamar `drawRichText` (sem render), conta as linhas e soma as alturas fixas (padding, título, corpo, data, assinatura) para calcular o `boxH`.

14.4 Paginação automática

A variável `y` rastreia a posição vertical atual no documento. Antes de desenhar cada recibo, o código verifica: **if (`y + boxH > PAGE_H - MB`)**. Se o recibo não couber na página corrente, `doc.addPage()` é chamado e `y` é resetado para a margem superior. Isso permite um documento com qualquer número de funcionários, distribuídos automaticamente nas páginas.

14.5 Funcionários com saldo zero são ignorados

Antes de gerar os recibos, o código filtra a lista de funcionários: apenas aqueles com **total > 0** recebem um recibo. Isso evita recibos de R\$0,00 para funcionários que não trabalharam na semana selecionada.

15. Backup e Restauração

O sistema de backup exporta **todos os dados do app** como um arquivo JSON e permite reimportar em qualquer dispositivo ou instalação.

15.1 Estrutura do backup

O arquivo JSON de backup tem o seguinte schema:

```
{
  "version": 1,
  "exportedAt": "2026-05-03T10:00:00.000Z",
  "semanas": [ /* array de semanas */ ],
  "pontos": [ /* array de pontos */ ],
  "employees": [ /* array de funcionarios */ ],
  "settings": { empresa, cnpj, endereco, cidade }
}
```

15.2 Validação na importação

A função **parseBackupFile(file)** lê o arquivo com **FileReader** e valida cada seção antes de aceitar o backup:

- Semanas: **id** string, **numero** number, **dataInicio** string, **dias** array
- Pontos: **id** string, **numero** number, **dataInicio** string, **dias** objeto
- Employees: **id** string, **name** string

Se qualquer validação falha, uma exceção é lançada com mensagem legível pelo usuário e o backup não é aplicado. Isso protege contra importação de arquivos corrompidos ou de versões incompatíveis.

15.3 Importação substitui todos os dados

A importação chama **replaceAll()** em cada store, substituindo completamente os dados existentes. Não há merge parcial. Isso é intencional: o backup é uma fotografia completa do estado do app em um ponto no tempo, e importar um backup deve restaurar exatamente esse estado.

16. PWA e Modo Offline

O app é uma **Progressive Web App (PWA)** configurada para funcionar 100% offline após a primeira instalação no dispositivo.

16.1 vite-plugin-pwa + Workbox

O plugin **vite-plugin-pwa** injeta automaticamente um service worker gerado pelo **Workbox**. O service worker é registrado em modo **autoUpdate**: quando um novo deploy é detectado, o SW é atualizado automaticamente na próxima visita.

16.2 Estratégia de cache — precache total

A configuração **globPatterns: ['**/*.js,css,html,svg,png,ico,woff2']** instrui o Workbox a incluir *todos* os assets no precache durante a instalação do SW. O app não usa cache runtime (`runtimeCaching: []`) porque não faz nenhuma requisição de rede — todos os dados são locais.

DECISÃO: Precache total em vez de cache-first ou network-first

Por quê: O app não consome APIs externas. Todos os dados estão em localStorage. A única coisa que precisa de cache são os arquivos estáticos do bundle. Precache total na instalação garante que, após a primeira visita com internet, o app funciona offline indefinidamente.

Impacto: Zero requisições de rede em uso normal. Funciona sem qualquer conectividade.

16.3 Dev com HTTPS — basicSsl

O plugin **@vitejs/plugin-basic-ssl** é ativado apenas em modo de desenvolvimento (**command === 'serve'**). Gera um certificado auto-assinado para o servidor Vite, permitindo instalar o app como PWA em um celular conectado à mesma rede Wi-Fi — browsers móveis exigem HTTPS para instalar service workers.

16.4 Web App Manifest

O manifest configura:

display: standalone	Remove a barra de URL do browser — parece app nativo
orientation: portrait	Trava em retrato — layout otimizado para mobile
background_color / theme_color	Cor #1c1715 (marrom escuro) — splash screen e barra de status
lang: pt-BR	Locale correto para o instalador do Android

17. Distribuição Android (Capacitor)

Para distribuir o app como um **APK Android** instalável offline sem Google Play Store, o projeto usa **Capacitor** do time Ionic.

17.1 Por que Capacitor e não PWABuilder

PWABuilder gera um APK que é um **Trusted Web Activity (TWA)** — uma cápsula Android que abre a URL do app hospedada na internet. Se não houver internet, o app não carrega. Para uso em fábrica sem wi-fi, isso é inaceitável.

Capacitor **embute os arquivos do build** dentro do APK. O WebView do Android serve os arquivos diretamente da memória do dispositivo. Não há requisição de rede, nunca.

DECISÃO: Capacitor em vez de PWABuilder ou Tauri

Por quê: PWABuilder: APK depende de URL ativa na internet (TWA). Tauri Android: ainda experimental em maio/2026, docs incompletos. Capacitor: maduro, suportado pelo time Ionic desde 2019, build nativo estável.

Impacto: APK de ~5 MB que roda 100% offline em qualquer Android 7+.

17.2 Fluxo de build do APK

Após instalar Android Studio, o processo é:

1	<code>npm install @capacitor/core @capacitor/cli @capacitor/android</code>	Instala pacotes Capacitor
2	<code>npx cap init FabricaLog com.cedan.fabricalog --web-dir dist</code>	Inicializa config, define bundle ID
3	<code>npx cap add android</code>	Cria a pasta android/ com projeto Android Studio
4	<code>npm run build</code>	Gera o bundle web em dist/
5	<code>npx cap sync</code>	Copia dist/ para android/app/src/main/assets/public
6	<code>npx cap open android</code>	Abre Android Studio
7	Build > Generate Signed Bundle/APK	Gera o APK assinado para instalação

17.3 Atualização do app

Quando o código do app muda, o processo de atualização é:

- **npm run build** — gera novo bundle em dist/
- **npx cap sync** — sincroniza os arquivos para o projeto Android

- Re-gerar o APK no Android Studio e reinstalar no dispositivo

Não é necessário rodar **npx cap add android** novamente — a pasta **android/** é gerada uma única vez. Apenas sync + build do Android.

17.4 Dados persistem entre versões

O **localStorage** do **WebView** do **Capacitor** persiste entre reinstalações do **APK** (desde que o usuário não desinstale o app). O prefixo **fabricalog_v1_** garante que uma nova versão com schema diferente não corrompa dados antigos.

17.5 Assinatura do APK com Keystore

O **APK** de release é assinado com um keystore **RSA 2048-bit** criado com **keytool**. As credenciais ficam em **android/gradle.properties** (não versionado). O **build.gradle** referencia as credenciais via **signingConfigs.release**. O comando **./gradlew assembleRelease** produz **app-release.apk** (sem -unsigned), pronto para instalação direta. Antes de cada build, remover **android/app/build/intermediates** para evitar **AccessDeniedException** do **OneDrive**.

18. Módulo Cargas

O módulo Cargas registra os carregamentos diários de tijolos. A tela principal navega por data (←/→) e lista os carregamentos do dia com um resumo (total de carregamentos, milheiros, empresa, externo).

18.1 Estrutura de um carregamento

Campos: **nome/destino** (texto), **qtd** (milheiros), **tipo** ('empresa' | 'externo'), **destino** ('venda' | 'estoque' | 'galpao'), **inicio** (HH:MM), **fim** (HH:MM), **obs** (textarea).

18.2 Tipos de caminhão

Dois tipos: **Empresa** (caminhão próprio da empresa) e **Externo** (caminhão do cliente). Caminhões externos implicam venda — sem campo de destino. Caminhões da empresa exigem seleção de destino.

18.3 Destino (somente para caminhão da empresa)

Três opções: **Venda** (produção vai para venda), **Estoque** (vai para estoque), **Galpão** (produto fica no galpão, aguardando). O campo **destino** só é exibido e salvo quando **tipo** === 'empresa'. Quando tipo é alterado para 'externo', o destino é limpo automaticamente.

18.4 Duração calculada

Se início e fim estiverem preenchidos, a duração é exibida automaticamente no formulário e no card (ex: "2h 15min"). Calculada como diferença em minutos entre fim e início.

18.5 useCargaStore

Store com padrão idêntico aos outros stores. Operação adicional **byDate(iso)** que filtra carregamentos pela data ISO selecionada.

19. Banco de Dados — Supabase

O FábricaLog usa uma arquitetura **offline-first**: o `localStorage` do dispositivo é a fonte primária de verdade e o Supabase (PostgreSQL gerenciado) funciona como camada de nuvem para backup automático, sincronização entre dispositivos e recuperação de dados em caso de reinstalação do app.

19.1 Arquitetura Offline-First

Toda operação do usuário escreve imediatamente no `localStorage`. O Supabase é escrito em segundo plano com `debounce` de 3 segundos. O app funciona completamente sem internet — o sync para a nuvem ocorre de forma silenciosa quando há conexão.

Fluxo de dados:

- Usuário edita dado → store atualiza `localStorage` imediatamente
- 3 segundos após a última edição → **`pushToCloud()`** envia tudo para o Supabase
- Ao abrir o app (com internet) → **`pullFromCloud()`** baixa dados da nuvem
- Merge automático: vence o item com **`updatedAt`** mais recente

19.2 Autenticação

Usa o **Supabase Auth** com email e senha. Não há registro público — as contas são criadas manualmente no painel do Supabase pelo administrador.

Fluxo de autenticação no App.jsx:

- **`supabase.auth.getSession()`** é chamado ao montar o componente. Se há sessão ativa, o usuário é autenticado sem passar pela tela de login.
- **`supabase.auth.onAuthStateChange()`** escuta mudanças de sessão em tempo real (ex: sessão expirada em outro dispositivo).
- Enquanto a sessão está sendo verificada, **`user === undefined`** — o app exibe "Carregando...". Após verificação: **`null`** → tela de login; objeto User → app normal.
- **LoginScreen.jsx**: formulário de email + senha. Em caso de erro retorna "Email ou senha incorretos." sem expor detalhes internos.
- **`handleLogout()`**: chama **`supabase.auth.signOut()`** e reseta user para `null`.

19.3 Schema do Banco de Dados

Uma única tabela **`app_data`** no PostgreSQL do Supabase armazena **todos os dados do app em colunas JSONB**. Cada usuário tem exatamente uma linha.

Estrutura da tabela `app_data`:

- **`user_id`** (UUID, PRIMARY KEY) — referencia `auth.users.id` do Supabase Auth
- **`semanas`** (JSONB) — array de semanas de produção
- **`pontos`** (JSONB) — array de folhas de ponto
- **`employees`** (JSONB) — array de funcionários
- **`settings`** (JSONB) — configurações da empresa (nome, endereço, etc.)
- **`forno`** (JSONB) — estado atual das câmaras dos fornos
- **`carregamentos`** (JSONB) — array de registros de carga
- **`updated_at`** (TIMESTAMPTZ) — timestamp ISO da última sincronização

SQL de criação da tabela:

```
CREATE TABLE app_data (  
  user_id      UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  semanas      JSONB DEFAULT '[]'::jsonb,  
  pontos       JSONB DEFAULT '[]'::jsonb,  
  employees    JSONB DEFAULT '[]'::jsonb,  
  settings     JSONB DEFAULT '{}'::jsonb,  
  forno        JSONB DEFAULT '{}'::jsonb,  
  carregamentos JSONB DEFAULT '[]'::jsonb,  
  updated_at   TIMESTAMPTZ DEFAULT now()  
);  
  
-- Row Level Security: cada usuário vê apenas seus próprios dados  
ALTER TABLE app_data ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY "Users can read own data"  
  ON app_data FOR SELECT  
  USING (auth.uid() = user_id);  
  
CREATE POLICY "Users can upsert own data"  
  ON app_data FOR ALL  
  USING (auth.uid() = user_id)  
  WITH CHECK (auth.uid() = user_id);
```

19.4 Push — Envio para a Nuvem

Implementado em `src/lib/sync.js` · função `pushToCloud()`.

Dispara com **debounce de 3 segundos** após qualquer mudança nos stores (semanas, pontos, carregamentos, employees, settings, forno). O debounce evita múltiplas requisições por keystroke — apenas a última versão consolidada é enviada.

Usa **upsert** com conflito em **user_id**: se a linha já existe, substitui; se não existe, insere. Isso garante que a tabela tenha sempre exatamente uma linha por usuário.

- Guarda indicador visual **syncing: true** durante o envio (aparece na Sidebar)
- Push só ocorre se **user !== null** — nunca tenta sincronizar sem autenticação
- Em caso de erro de rede, os dados permanecem íntegros no localStorage

19.5 Pull — Recebimento da Nuvem

Implementado em `src/lib/sync.js` · função `pullFromCloud()`. Chamada em quatro momentos:

- **Na inicialização**: logo após verificar a sessão ativa
- **Ao voltar ao app**: evento **visibilitychange** (ex: usuário alterna apps no celular)
- **Reconexão de rede**: evento **online** do browser (Capacitor propaga)
- **Polling**: a cada **60 segundos** enquanto o app está visível

Após o pull, os dados recebidos são mesclados com os locais via **mergeByUpdatedAt()**. O resultado substitui o store local com **replaceAll()**.

19.6 Merge — Conflito Entre Dispositivos

Implementado em **src/utils/syncMerge.js** · função **mergeByUpdatedAt(local, remote)**.

Estratégia **last-write-wins por item**: para cada item com o mesmo ID, vence o que tiver o campo **updatedAt** mais recente (string ISO 8601 comparada lexicograficamente). Itens sem **updatedAt** são tratados como mais antigos.

Regras do merge:

- Item existe só local → mantido
- Item existe só remoto → adicionado ao local
- Item existe nos dois → vence o com **updatedAt** maior
- Não há deleção propagada — itens deletados localmente ressurgem no próximo pull

A função utilitária **now()** no mesmo arquivo retorna **new Date().toISOString()** para uso consistente nos stores.

19.7 Backup Automático Diário

Após cada pull bem-sucedido, o app verifica se já foi feito um backup JSON hoje (chave **fabricalog_auto_backup_date** no **localStorage**). Se não, aguarda **4 segundos** para os stores estarem populados e gera silenciosamente um arquivo **.json** de backup completo no dispositivo.

- Executa no máximo uma vez por dia por dispositivo
- Silent: não exibe notificação ao usuário
- Em caso de erro, **autoBackupDone.current** é resetado para permitir nova tentativa
- Complementa o Supabase — cópia local mesmo se o banco cloud estiver indisponível

19.8 Arquivos Relacionados

src/lib/supabase.js — Inicializa o cliente Supabase com as variáveis de ambiente. Exporta o singleton **supabase** usado em todo o app.

src/lib/sync.js — Funções **pushToCloud()** e **pullFromCloud()**. Único arquivo que conhece a estrutura da tabela **app_data**.

src/utils/syncMerge.js — Algoritmo de merge **mergeByUpdatedAt()** e utilitário **now()**. Pura função, sem dependências externas, 100% testável.

src/modules/auth/LoginScreen.jsx — Tela de login com email/senha. Não tem registro — contas são criadas manualmente no Supabase Dashboard.

19.9 Variáveis de Ambiente

O cliente Supabase requer duas variáveis no arquivo **.env.local** (não commitado no repositório):

- **VITE_SUPABASE_URL** — URL do projeto Supabase (ex: `https://<project-id>.supabase.co`)
- **VITE_SUPABASE_ANON_KEY** — chave pública anon/publishable. Segura para expor no cliente pois o Row Level Security garante que cada usuário só acessa seus próprios dados.

Atenção: o **service_role key** (chave de admin) NUNCA deve ser usado no app cliente. Apenas no backend/migrations.

19.10 Row Level Security (RLS)

O Supabase aplica RLS em nível de banco de dados. Mesmo que o **anon key** seja exposto publicamente, um usuário autenticado só consegue ler e escrever a linha onde **user_id = auth.uid()**. É impossível acessar dados de outro usuário via API pública.

As policies necessárias estão documentadas no SQL da seção 19.3. Devem ser criadas no Supabase Dashboard → Authentication → Policies após a criação da tabela.

Documentação gerada em maio de 2026. FabricaLog v1.2 — Forno CEDAN, Bela Cruz – CE.